WEB-FAZENDA

Regras de Negócio do Sistema

Documento técnico em português

Projeto: Web-Fazenda

Escopo: backend NestJS, PostgreSQL e módulo Estações, Clima e Animais

Base analisada: branch `main`, após a consolidação das correções

Data: 18 de outubro de 2026

1. Objetivo e critério de elaboração

Este documento registra as regras de negócio identificadas no código-fonte atual do Web-Fazenda. A finalidade é apresentar ao professor o comportamento esperado e as restrições efetivamente implementadas no sistema.

> **Critério:** uma regra só é apresentada como implementada quando existe validação em serviço, guarda de autorização, transação, restrição de banco de dados, DTO ou teste correspondente. Regras que possuem apenas campos preparados, mas ainda não possuem fluxo executor, são indicadas como “parcialmente implementadas” ou “ponto de atenção”.

O sistema é uma API para gestão acadêmica e agropecuária. Ele conecta usuários, perfis e turmas a fazendas, talhões, culturas, plantios, colheitas, estações, temporadas, eventos climáticos, rebanhos, alimentação, estoque e produção.

2. Atores e controle de acesso

Ator	Regras de atuação
Administrador	Usuário com perfil `admin`. O guard de permissões concede acesso quando o usuário é administrador, independentemente da lista individual de permissões.
Professor	Pode operar recursos autorizados apenas dentro das próprias turmas. A titularidade é conferida pelo `professor_id` da turma.
Aluno	Visualiza e opera dados vinculados à própria turma ou às próprias fazendas, conforme o domínio. Listagens são filtradas para impedir exposição de dados de outras turmas/fazendas.

Usuário não autenticado	Não acessa endpoints protegidos por JWT. Quando um endpoint exige permissões e não há usuário autenticado, o acesso é negado.

2.1 Regras transversais de autorização

- A API usa autenticação Bearer JWT; o token carrega identificador, e-mail, perfil, permissões e turma do usuário.
- A API usa autenticação por token JWT; o token carrega identificador, e-mail, perfil, permissões e turma do usuário.
- Os endpoints protegidos combinam `JwtAuthGuard` e `PermissionsGuard`.
- Quando um endpoint exige mais de uma permissão, todas as permissões exigidas devem estar presentes.
- O sistema aplica limitação global de requisições: 100 requisições por janela de 60 segundos.
- A senha nunca é devolvida no cadastro de usuário; ela é armazenada com hash BCrypt.

3. Identidade, perfis e permissões

3.1 Autenticação

- O login exige e-mail e senha válidos.
- E-mail inexistente e senha incorreta produzem a mesma mensagem de credencial inválida, evitando diferenciar usuários cadastrados.
- O cadastro rejeita e-mail já utilizado.
- Se o perfil não for informado no cadastro, o usuário recebe o perfil `aluno`, quando este existir.
- O usuário pode ser associado a uma turma; a ausência de turma é permitida para perfis administrativos.

3.2 Perfis e permissões

- Nome de perfil e nome de permissão são únicos.
- Uma associação perfil-permissão não pode ser repetida.
- Permissões são organizadas por módulo e ação, por exemplo: `fazendas:criar`, `plantios:colher` e `temporadas:avancar`.
- Excluir um perfil ou permissão remove suas associações na tabela de vínculo; excluir um perfil atribuído a usuário é restringido pelo banco.

4. Turmas e matrícula de alunos

- O código da turma é obrigatório e único.

- O professor só pode consultar, atualizar, remover ou matricular alunos em turmas das quais é responsável.
- O aluno só pode acessar a turma em que está matriculado. A matrícula é reconhecida pelo vínculo da turma, pelo `turma_id` do usuário ou por ambos.
- Um aluno não pode ser matriculado duas vezes na mesma turma.
- A quantidade de alunos não pode atingir valor superior ao limite `max_alunos`; o padrão de banco é 40 quando não informado.
- A relação turma-aluno é removida em cascata quando a turma ou o usuário é excluído.
- Excluir uma turma também remove temporadas e eventos vinculados, conforme as restrições de integridade.

5. Culturas, fazendas e talhões

5.1 Culturas

- O nome da cultura é único.
- Uma cultura informa estações permitidas, dias de crescimento, área e água por unidade, rendimento e valor por unidade.
- Uma cultura não pode ser removida quando já possui histórico de plantio.

5.2 Fazendas

- A fazenda pode ser vinculada a uma turma e/ou a um usuário; os vínculos precisam existir.
- `agua_disponivel` começa com o valor de `agua_total`.
- Ao aumentar ou reduzir a água total, a quantidade já consumida é preservada. A água total não pode ficar menor que o volume já utilizado.
- A área total e a área livre são calculadas a partir dos talhões associados.
- Uma fazenda com histórico de plantios não pode ser excluída.
- Fatores de água e produtividade começam em 1 e podem ser alterados por estações e eventos climáticos.

5.3 Talhões

- O talhão deve pertencer a uma fazenda existente.
- O código do talhão é único dentro da fazenda; códigos iguais em fazendas diferentes são permitidos.
- A área de um talhão plantado não pode ser alterada.
- Um talhão com histórico de plantios não pode ser excluído.
- Ao criar um plantio, o talhão passa para o estado `plantado`; após a colheita, retorna a `livre`.

6. Plantio e colheita

6.1 Criação de plantio

- O talhão, a cultura e a fazenda devem existir.
- Um talhão que já está plantado não pode receber outro plantio.
- A cultura precisa ser compatível com a estação atual da fazenda.
- A área utilizada é calculada por `quantidade × area_por_unidade` e não pode superar a área do talhão.
- A água utilizada é calculada por `quantidade × agua_por_unidade` e não pode superar a água disponível da fazenda.
- A data de plantio padrão é a data atual quando não informada.
- A data prevista de colheita é a data de plantio somada aos dias de crescimento da cultura.
- Se a mesma cultura foi plantada no ciclo anterior no mesmo talhão, o plantio recebe penalidade de rotação.
- A criação do plantio, alteração do talhão e débito da água ocorrem na mesma transação.

6.2 Colheita

- Um plantio inexistente não pode ser colhido.
- Um plantio já colhido não pode ser colhido novamente.
- A data da colheita não pode ser anterior à data do plantio.
- Colheita antecipada gera perda proporcional; quando ocorre antes da metade do crescimento, a perda é total.
- A penalidade de rotação reduz o aproveitamento em 20%.
- A quantidade colhida e o valor estimado são calculados com base na quantidade plantada, rendimento/valor da cultura e aproveitamento final.
- A criação da colheita, atualização do plantio e liberação do talhão são atômicas.
- Um agendador verifica periodicamente plantios cuja data prevista chegou e os marca como prontos para colher.

```
quantidade_colhida = quantidade_plantada × rendimento_por_unidade ×
```

```
aproveitamento
```

```
valor_estimado = quantidade_plantada × valor_por_unidade × aproveitamento
```

7. Estações e temporadas

7.1 Estações

- O nome da estação é único e pertence ao conjunto Primavera, Verão, Outono e Inverno.
- A ordem deve ser inteira e maior ou igual a 1; não pode haver duas estações com a mesma ordem.
- Os fatores de produtividade e água devem ser positivos e, por padrão, valem 1.

7.2 Temporadas

- A temporada pertence a uma turma e a uma estação existentes.
- O número da temporada é sequencial por turma.
- A data final não pode ser anterior à data inicial.
- Uma temporada nova começa como `PLANEJADA`.
- Uma temporada planejada pode avançar para `PROCESSANDO`, desde que não exista outra temporada ativa para a mesma turma.
- Temporadas `PROCESSANDO` ou `ENCERRADA` não podem ser avançadas novamente pelo mesmo fluxo.
- Somente o professor responsável pela turma pode gerenciar temporadas; administradores não sofrem essa restrição de titularidade.
- Na ativação, a temporada muda para `ATIVA` e recebe data de início quando necessário.
- Ao encerrar, a temporada é marcada como `ENCERRADA`; se houver próxima temporada planejada, ela é ativada. Caso contrário, cria-se uma temporada ativa na próxima estação, voltando à primeira quando o ciclo termina.
- O banco mantém um índice único parcial para impedir mais de uma temporada ativa por turma.

Estado	Significado
PLANEJADA	Cadastrada, mas ainda não iniciada.
PROCESSANDO	Em transição para ativação ou processamento do ciclo.
ATIVA	É a temporada corrente da turma e pode receber eventos climáticos.
ENCERRADA	Ciclo finalizado; não pode ser avançado novamente.

8. Eventos climáticos

- Evento climático só pode ser criado em temporada `ATIVA`.

- Professor só pode criar ou encerrar eventos das próprias turmas.
- O multiplicador não pode ser 1, pois não produziria efeito.
- O alvo deve ser compatível com o tipo do evento.

Tipo	Alvo permitido	Direção do multiplicador
SECA	ÁGUA	Reduzir: menor que 1
CHUVA_FORTE	ÁGUA	Aumentar: maior que 1
GEADA	PRODUTIVIDADE	Reduzir: menor que 1
ONDA_CALOR	ÁGUA ou PRODUTIVIDADE	Reduzir: menor que 1
BOA_SAFRA	PRODUTIVIDADE	Aumentar: maior que 1

- Ao ativar o evento, o multiplicador é aplicado a todas as fazendas da turma, dentro de uma transação.
- Ao encerrar o evento, o efeito é revertido pela divisão pelo mesmo multiplicador.
- O vínculo entre fazenda e evento evita aplicar duas vezes o mesmo evento na mesma fazenda.
- Evento encerrado não pode ser encerrado novamente.
- A exclusão da temporada remove eventos dependentes; o usuário que criou o evento pode ser removido sem apagar o evento, pois o vínculo é definido como nulo.

9. Espécies, rebanhos e alimentação

9.1 Espécies

- O nome da espécie deve ter entre 2 e 60 caracteres e é único.
- Produto e unidade do produto são obrigatórios.
- Consumo de ração, produção por animal e preço base não podem ser negativos.
- A taxa de crescimento deve estar entre 0 e 1.
- Espécie inativa não aceita novos animais.

9.2 Rebanhos

- O rebanho vincula uma fazenda a uma espécie.
- Ao adicionar animais à mesma combinação fazenda-espécie, a quantidade existente é somada em vez de criar outro rebanho.

- Um novo rebanho começa com saúde `SAUDAVEL`.
- Alunos só podem criar ou alimentar rebanhos de suas próprias fazendas.
- Professores só podem operar rebanhos de fazendas ligadas às suas turmas.
- A listagem de alunos é restringida às próprias fazendas; filtro por fazenda de terceiro retorna conjunto vazio.

9.3 Alimentação

- Não é possível alimentar rebanho com quantidade de animais menor ou igual a zero.
- Quando a temporada não é informada, o sistema tenta localizar a temporada ativa da turma da fazenda.
- Um rebanho só pode ser alimentado uma vez por temporada.
- A ração necessária é `quantidade_de_animais × consumo_por_animal`.
- A ração fornecida é limitada ao menor valor entre estoque disponível e ração necessária.
- Se a ração fornecida for menor que a necessária, o rebanho fica `SUBNUTRIDO`; caso contrário, fica `SAUDAVEL`.
- Consumo de estoque, atualização da saúde e registro da alimentação são executados na mesma transação.

```
proporcao_atendida = racao_fornecida ÷ racao_necessaria
```

10. Estoque e produção

- Cada fazenda possui no máximo um registro de estoque.
- O estoque de ração inicia em zero e nunca pode ser consumido abaixo de zero.
- Produtos são armazenados por nome e quantidade em estrutura JSON.
- O crédito de produto soma a quantidade ao saldo existente; quando o estoque ainda não existe, ele é criado.
- Os modificadores climáticos são persistidos por fazenda e evento, com alvo restrito a água ou produtividade.
- Estoque, modificadores e rebanhos são removidos em cascata quando a fazenda é removida.

> **Ponto de atenção:** as entidades e parâmetros de produção estão modelados (produto, unidade, produção por animal, preço e fator de produtividade), e o sistema possui consulta de produções. Entretanto, na versão analisada não foi localizado um fluxo completo que gere automaticamente a produção ao avançar/encerrar uma temporada e credite o estoque de produtos. Essa regra deve ser confirmada ou implementada antes de apresentar a produção automática como concluída.

11. Integridade, transações e persistência

- UUIDs são gerados pelo PostgreSQL com suporte a `pgcrypto`.
- As relações entre turmas, usuários, fazendas, temporadas, eventos, rebanhos, estoques, alimentações e produções são protegidas por chaves estrangeiras.
- Dependências acadêmicas essenciais usam `CASCADE`, `RESTRICT` ou `SET NULL` conforme a natureza do vínculo.
- Criação de plantio, colheita, aplicação/reversão climática e alimentação usam transações para evitar estados parcialmente gravados.
- Os adaptadores de fazenda e estoque aceitam o cliente transacional ativo; assim, leituras e escritas relacionadas participam da mesma unidade de trabalho.
- Migrações foram verificadas em PostgreSQL 17 com aplicação, rollback total e reaplicação das 17 migrações.

12. Validações, respostas e consultas

- DTOs aplicam validação global de tipos, UUIDs, enumerações, limites numéricos, datas e campos obrigatórios.
- Listagens usam paginação, filtros e metadados de página.
- IDs de recursos são tratados como UUID quando definido pelo endpoint.
- Erros de inexistência retornam 404; conflitos de unicidade ou repetição retornam 409; falta de permissão retorna 403; credencial inválida retorna 401; regra de domínio inválida retorna 400 ou 422 conforme o caso.
- A especificação OpenAPI está disponível em `/api/docs` e `/api/docs-json`, com 36 caminhos e 45 esquemas na versão validada.

13. Matriz resumida de permissões

Módulo	Ações principais
Usuários	criar, listar, visualizar, editar, deletar
Perfis	criar, listar, visualizar, editar, deletar e vincular permissões
Permissões	criar, listar, visualizar, editar, deletar
Turmas	criar, listar, visualizar, editar, deletar e matricular alunos
Culturas	criar, listar, visualizar, editar, deletar

Fazendas	criar, listar, visualizar, editar, deletar
Talhões	criar, listar, visualizar, editar, deletar
Plantios	criar, listar, visualizar e colher
Estações/ temporadas	cadastrar, listar, visualizar, avançar e gerenciar ciclo
Clima	criar, listar, visualizar, encerrar e aplicar efeitos
Animais	espécies, rebanhos, alimentação e produção

14. Pontos de atenção para avaliação acadêmica

1. A produção animal automática precisa ser confirmada: o modelo está preparado, mas a geração automática não aparece completa no fluxo analisado.
2. Os testes unitários e de integração validam as regras principais, porém os testes E2E dependentes de banco são opt-in e ficam desabilitados por padrão.
3. A base funcional está validada, mas ainda existem arquivos legados com divergências de formatação Prettier; isso não altera as regras de negócio.
4. Antes da entrega final, recomenda-se demonstrar no ambiente de homologação: login, matrícula, criação de fazenda, plantio, colheita, temporada, evento climático, alimentação e rollback de transação.

15. Fontes principais no código

Área	Arquivos de referência
Autenticação e autorização	`backend/src/auth/auth.service.ts`; `backend/src/auth/guards/permissions.guard.ts`; controllers com `@RequirePermissions`
Turmas e usuários	`backend/src/turmas/turmas.service.ts`; `backend/src/usuarios/usuarios.service.ts`; `backend/src/db/migrations/20261013030212_api_structure.ts`
Agricultura	`backend/src/fazendas/fazendas.service.ts`; `backend/src/talhoes/talhoes.service.ts`; `backend/src/plantios/plantios.service.ts`; `backend/src/plantios/plantios.scheduler.ts`
Estações e temporadas	`backend/src/estacoes-animais/estacoes/estacoes.service.ts`; `backend/src/estacoes-`

	`animais/temporadas/temporadas.service.ts`
Clima	`backend/src/estacoes-animais/eventos-climaticos/eventos-climaticos.service.ts`; `backend/src/estacoes-animais/ports/fazendas.adapter.ts`
Animais e alimentação	`backend/src/estacoes-animais/rebanhos/rebanhos.service.ts`; `backend/src/estacoes-animais/rebanhos/alimentacoes.service.ts`; `backend/src/estacoes-animais/ports/estoque.adapter.ts`
Integridade do banco	`backend/src/db/migrations/20261012115900_enable_pgcrypto.ts`; `backend/src/db/migrations/20261018120000_create_estoques_e_modificadores.ts`; `backend/src/db/migrations/20261018130000_add_estacoes_animais_integrity_constraints.ts`

Conclusão: o Web-Fazenda possui regras consistentes de autenticação, autorização por perfil, gestão acadêmica, ciclo agrícola, temporadas, clima, rebanhos, alimentação, estoque e integridade transacional. O principal ponto funcional a confirmar é a automação da geração de produção animal, que está parcialmente modelada, mas não aparece completa no fluxo de execução analisado.